# Effect4 and web standards semantics

**A research basis for the next reification contracts**

2 September 2026. Prepared for the Effect4 project owner and future contract authors. Scope: the ten supplied questions, current local repositories, the pinned Effect 4 runtime, relevant standards, original research, and fresh local measurements.

The strongest direction is a shared, explicit model of asynchronous execution with separately proved bridges. Sharing a JavaScript host does not establish agreement between Effect and a web API. The research also rules out three shortcuts: returned error values are not abrupt failure; a finalizer is not an ordinary JavaScript `finally` block around `yield*`; and a stream high-water mark is not a hard memory limit.

This report recommends contracts. It does not amend existing rulings, freeze public declarations, authorize cutover, or claim new end-to-end correctness proofs. Proposed laws below are distinguished from existing theorems and finite experiments.

## Answers at a glance

| Question | Research conclusion |
|---|---|
| 1. Shared event loop | Share promise/job state and a host profile; retain Effect scheduling as a separate policy layer. |
| 2. Observational equivalence | Compare invocation protocols under an explicit observation mask, with state relations and divergence obligations. |
| 3. Errors | Keep returned error data and aborting error operations distinct; specify their conversion. |
| 4. Open unions | Use membership evidence over existing morphisms, with a declared normal form and duplicate policy. |
| 5. Scoped operations | Keep first-order child IDs; prove delimitation, captured-environment and execution laws. |
| 6. Decisions | Unify typed requests and replay answers; preserve internal choice, external input and policy distinctions. |
| 7. Flow lowering | Begin with a bounded source profile and a simulation that allows finite internal steps. |
| 8. Cleanup and abort | Prove ownership and at-most-once invocation; add progress assumptions for eventual completion. |
| 9. Corpus priorities | Prioritize generators, services/layers and errors, then ordinary loops and resources; retain sample caveats. |
| 10. Streams bridge | Use M1 for chunks/outcomes, M2 for ordering, and a separate backpressure invariant. |

# The current ground

The copied plans are behind the current checkout. Effect4's current package file names Effects v0.2.0 and a separate TypeScript package. Signature morphisms, monad transport, operation families and generic Flow already exist in Effects. Some routers still describe the earlier, single-dependency v0.1.0 arrangement. The local override file selects sibling checkouts, while the recorded dependency manifest remains stale. These are local verification results, not proof of a reproducible clean download. Local evidence L1.

| Component | Observed revision or authority |
|---|---|
| Effect4 checkout | `6c9f479cf927dfcd0dad14892ea6a7b79f835a03` |
| Effects checkout and intended pin | `fa3cdc96c0dc631e73174aba8a2175a0ef11125a` |
| TypeScript package intended pin | `8f17b881950476c211853fa76a6b467c037f0c76` |
| Standalone WHATWG checkout | `00f8f2d3db311e687b84eb026fe4f9f4908f952d` |
| Runtime target | `effect@4.0.0-rc.112`; upstream `2600f62f4532026928454dcea8d1c48557b3f942` |
| Streams standard | `b9ba9f49d95b4280be0dc2372377a006c3a91c18` |

Effect4 has implemented frame and scope models. Its general configuration, step relation, structured regions and source-to-target simulation remain declaration-free stubs. The target currently renders a first-order straight-line script. Effects' own claim boundary says the newer morphism/transport/family additions preceded their independent contract packets. Existing theorem statements can be used; assurance closure must not be inferred. Local evidence L1-L3.

The earlier Foldlab record remains useful. Its program-carrier and handler reviews identify finite-program limits, failure-state loss and missing automatic membership. Its narrow application AST decision is historical scope, not a prohibition on Effect4's later Flow project. `THE-ALGEBRA.md` explicitly labels itself unratified research. Local evidence L4.

# 1. What event-loop model should both refine?

**Recommendation: a small single-agent asynchronous core, parameterized by its host, with Effect's scheduler above it.** The ECMAScript promise-job queue, HTML task scheduling and Effect fiber scheduling must remain distinct.

ECMAScript requires promise jobs to run in the order of the scheduling `HostEnqueuePromiseJob` calls. HTML has multiple task queues, selects among them under its processing model, and takes a runnable task from the selected queue. Its microtask queue is separate. FIFO promise jobs therefore do not imply a deterministic whole event loop. ECMA-262, HostEnqueuePromiseJob, HTML, event loops.

At the pinned Effect version, `MixedScheduler` defaults to asynchronous dispatch using `setImmediate` where available and a zero-delay timer otherwise. Its synchronous mode selects a promise-microtask helper. It groups tasks by priority, with FIFO order within a priority, and drains a snapshot batch. A model that simply appends every fiber step to one promise queue would miss those choices. Local evidence L5.

The proposed configuration has four parts:

- Promise records: stable identities, pending/settled status, result or rejection, and reaction registrations. Include resolution locks and adoption state, or explicitly exclude thenables and promise adoption from the initial profile.

- Job state: current execution, FIFO microtasks, checkpoint activity and first-order callback identities.
- Host profile: task sources, eligible queues, runnable conditions, external arrivals and scheduling rules. HTML and Node are separate profiles.
- Library state: Effect priority buckets, operation budget and parked fibers; Streams objects, locks, pending requests and queues.

A promise resolved to another pending promise is still pending, but a later rejection cannot override that resolution. A pending/settled flag alone misses this distinction. ECMA-262, Promise objects.

**Jobs can be both an interface and state.** A `Jobs` signature may request promise creation, reaction registration, settlement and enqueueing. Its handler updates configuration. There is no need to choose between an inhabited signature and stored queue state. An empty signature is appropriate only if no program performs any of those operations through that interface.

Loring, Marron and Leijen's 2017 semantics is a useful precedent for parameterizing asynchronous scheduling. Its abstract scheduling admits behaviors beyond a particular Node implementation; it is not current HTML or Node authority. The transferable idea is an explicit policy boundary. Semantics of Asynchronous JavaScript, sections 2.3-2.3.2.

### The proof that sharing does not supply

Suppose the common model allows visible results `{a,b}`. An Effect realization that allows only `{a}` and a Streams realization that allows only `{b}` both refine it, yet disagree. Common refinement does not in general entail mutual equivalence. Each bridge needs related initial states, the same observations, compatible decisions and a matching-step argument in the claimed direction. Reverse adequacy is additional work.

Start with one agent, no transfer and no shared memory. Treat foreign callbacks and identities through explicit boundaries. Prove queue order, single settlement, checkpoint behavior and preservation of well-formed state. Add fairness only to theorems that need eventual progress. An endlessly replenished microtask queue or nonterminating callback can prevent later task execution even when FIFO is respected.

For the wider standards effort, pure byte codecs and parsing atoms can proceed independently of this asynchronous core. The local stratified strategy already distinguishes value descriptions, pure atoms and stateful interfaces. A common foundation should connect these layers without making synchronous standards wait for a complete browser model. Local evidence L6.

## 2. Which equivalence relates Effect to a promise API?

**Recommendation: relate an invocation and interaction protocol, not a bare Effect value to an already-created promise.** Fix the provided services, start one execution, register observers, issue consumer operations and apply cancellation through a named adapter.

An Effect value can describe work that starts when run; a promise can already represent started or completed work. The adapter must state whether repeated invocations start fresh work or share it. It must also map successful values, typed failures, defects, interruption, combined causes and relevant identities. Omitting those choices hides observable differences. The fresh generator probe confirmed that its body had not run at construction. Local evidence L7.

WHATWG's registered masks are:

- **M1:** chunks delivered to the consumer and the terminal outcome: closed, errored with reason, cancelled with reason or aborted with reason.

- **M2:** M1 plus the order of consumer-observable promise settlements, including `ready`, `closed`, `desiredSize` reads and read results.

The M2 wording needs a precise event alphabet: `desiredSize` is a synchronous getter observation, and promise settlement differs from execution of a reaction registered with `.then`. Record those events separately before defining either projection. This is a proposed clarification, not an alteration of DB-04. Local evidence L6.

A finite Node experiment returned the same value through two paths. Adding one identity `.then` changed the observed order from `[result, marker]` to `[marker, result]`. This is a counterexample to treating every extra promise reaction as invisible. It does not establish all of M2 or browser behavior. Local evidence L7.

### Recommended theorem form

Define a relation between source state, adapter state and target state. For each relevant source transition, find target transitions with the same retained observations and related resulting state. Permit only finite stretches of invisible administrative steps. Add explicit terminal-result correspondence and a divergence condition; otherwise the target could remain silent forever while the source finishes.

That direction establishes source-step realization; it does not exclude extra target behaviors. Behavior preservation needs target-to-source trace inclusion, or explicit determinism and progress premises that make the forward argument sufficient. Equivalence requires the selected observations to agree in both directions, with matching interaction and divergence obligations.

Interaction Trees provides a model of weak equivalence, return relations and compositional interpretation. Choice Trees separates internal branching from external interaction, which matters when consumers can respond between choices. These are proof patterns to adapt, not existing proofs about Lean Flow. Interaction Trees, section 2, Choice Trees, sections 2-3.

Trace inclusion is enough for a clearly bounded trace claim. Substitution into interactive contexts may require a branching or contextual relation. Layered interpreter reasoning also requires the relevant monad and interpreter laws at each layer; sharing an interface is insufficient. Formal Reasoning about Layered Monadic Interpreters, sections 3-4.

## 3. Returned error answers or an error summand?

**Recommendation: preserve both meanings and specify the conversion.** An operation returning `Except E A` supplies ordinary answer data. An aborting error operation, with answer type `Empty`, supplies no ordinary answer. Neither design requires redefining the free-program algebra.

Web IDL distinguishes creating, throwing and returning an exception object, and rejecting a promise with a reason. It does not justify identifying all these behaviors. Callback exceptions and rejection reasons can carry arbitrary JavaScript values, so a closed enumeration describes only an admitted subset of standard-created failures. Web IDL, exceptions and JavaScript binding.

Fresh Lean witnesses demonstrate the difference: after receiving an error-valued reply, a program can recover and return `7`; an ambient failing interpretation aborts instead. Nullary `throw` absorbs a following bind. Catching a body also differs from catching the body plus subsequent work. Plotkin and Pretnar's account supplies the corresponding distinction between exception raising and handling. Handling Algebraic Effects, introduction and section 1. Local evidence L8.

The useful decomposition is:

| Boundary | Representation and obligation |
|---|---|
| Pure parser or codec | Explicit success/error data, with its exact failure predicate. |

| Boundary | Representation and obligation |
|---|---|
| Standard operation | Normal/abrupt completion when the standard distinguishes them. |
| Algorithm propagation | Checked conversion from abrupt completion to the intended aborting failure. |
| Catch | Delimited body and outer continuation; subsequent failures stay outside. |
| Foreign callback | Admitted JavaScript-value boundary, preserving the promised observations. |
| Effect target | Distinguish returning an error-shaped value successfully from `Effect.fail`. |

Current target operation rows expose answer spellings but no per-operation error field; method types render `Effect.Effect<Answer>`. Adding an error slot is a target contract decision. Rendering an `Except` reply directly as `Effect< A, E >` would be wrong unless the lowering implements the selected propagation semantics. Local evidence L3.

### When is Cause a lawful quotient target?

`Cause` is an error datatype. The fact that `Except Cause` supports lawful sequencing says nothing about a proposed quotient on cause values. Current rc.112 causes are ordered lists of annotated reasons, rather than the old sequential/parallel tree. Fresh witnesses show that reversing distinct failures can change `squash`, and equal squash results can conceal different defects. Local evidence L2, L8.

For a proposed projection `q`, first define its equivalence classes and retained observations. Prove that cause combination descends to those classes, every retained observation factors through `q`, and the admitted catches, cleanup and interruption operations respect the relation. Then prove the resulting computation translation respects return and bind. A squash projection may suit a terminal display boundary while being invalid inside a program that can inspect remaining reasons.

The first error packet should contain successful reply, returned error data, propagated error, recovery inside a scope, failure outside that scope, and body-state retention after failure. The last item preserves adopted DB-07 and the earlier Foldlab finding that discarded failure state breaks cleanup reasoning. Local evidence L4.

## 4. What open-union design, and what Lean cost?

**Recommendation: use a small membership interface over the existing signature morphisms, with a canonical shape and explicit evidence of inclusion.** Keep service requirement sets separate from runtime handler instances.

Swierstra's membership class searches a right-associated, list-shaped sum; the paper explicitly discusses failures for left-associated shapes. Kiselyov and Ishii's open union additionally supplies projection and removal of the leading effect. Koka's duplicate labels have deliberate meaning, so its row design is not evidence that two runtime handler instances may always be merged. Data Types a la Carte, section 4, Freer Monads, More Extensible Effects, section 2.2, Koka, section 2.4.

An arbitrary `Signature.Hom` need not be injective or preserve instance identity. The existing `codiag` collapses two copies of a signature. Fresh witnesses give those copies handlers returning `0` and `1`; their requests are distinguishable before collapse and identical afterward. Membership should retain its construction from inclusions, or expose the stronger laws required for an embedding. Local evidence L8-L9.

### Fresh measurements

The experiment used Lean 4.33.1 and Effects `fa3cdc96` on this Mac. Each file requested the first, middle and last member of a right-associated row. Times are medians of three fresh Lean processes and include startup and imports. Explicit terms were deeply nested morphism expressions, not an optimized generated-evidence design.

| Row width | Membership search, seconds | Explicit nested term, seconds |
|---|---|---|
| 8 | 0.2105 | 0.2093 |
| 16 | 0.2315 | 0.2144 |
| 32 | 0.2211 | 0.2502 |
| 64 | 0.2368 | 0.3049 |
| 128 | 0.2702 | 0.6297 |

Baseline startup/import time was 0.2182 seconds. Small differences near that baseline have little interpretive value. At width 256, default recursion limits rejected the row. Raising recursion depth to 2048 still left the deeper membership requests over the default instance-size limit of 128. Raising that limit to 1024 yielded a median of 0.3326 seconds; the explicit nested term with raised recursion depth took 3.0556 seconds. These are bounded front-end measurements, not runtime benchmarks or a universal scaling claim. Local evidence L9.

Other probes confirmed that this search works through `abbrev` row definitions and fails through an opaque-to-instance-search `def` in the tested case. Right-only search rejects left-associated rows. Operation universes combine by maximum; answer universes must match, with no implicit lift. The timing sample used universe-zero answers.

Evidence-passing implementations offer a different tradeoff: build canonical evidence vectors, then select the appropriate runtime handler by position. That literature concerns runtime selection, not Lean typeclass cost. Xie and Leijen, Generalized Evidence Passing for Effect Handlers, sections 2.5 and 3.3.

The next contract should state membership coherence, answer transport, injection/projection round trips, normalization preserving interpretation, residual handling and the duplicate-instance policy. Use unique service identity when requirements are deduplicated; give distinct instances distinct keys. Benchmark generated, shared evidence before choosing global limit increases.

## 5. Which account fits children represented by block IDs?

**Recommendation: retain the adopted first-order block-ID approach.** Scoped, latent and hefty accounts describe different problems; choosing one label does not settle all of them.

Scoped operations distinguish enclosed computation from subsequent outer work. A naive fork representation can wrongly attach the parent's following bind to the spawned child. Scoped syntax and its substitution laws address that distinction. Effect Handlers in Scope, section 11, Syntax and Semantics for Operations with Scopes.

Latent effects address deferred execution and the effect state carried to that execution. Hefty algebras organize higher-order interfaces and modular elaboration into lower-level effects. These ideas help specify the meaning of first-order child references; they do not require storing Lean functions in canonical

program data. Latent Effects for Reusable Language Components, section 3, Hefty Algebras, sections 3-5.

| Construct | Contract that matters |
|---|---|
| `scoped` and `catch` | Body boundary, outer continuation and effect forwarding. |
| `acquireRelease` | Successful acquisition, registration, captured context and later release. |
| fork | Child entry/captures, ownership, parent continuation and supervision. |
| race | Spawned competitors, winner criterion, loser interruption and cleanup. |
| Delayed callback | Which environment is captured, which state is read later, and permitted invocation count. |

A child ID alone says where code lives, not what environment it receives or how often it runs. The next region packet should therefore freeze lookup/type validity, explicit captures, lexical service ownership, body/continuation delimitation, forwarding, cancellation behavior and observation-preserving elaboration. The scoped-calculus literature treats inner and outer continuations and forwarding explicitly. A Calculus for Scoped Effects and Handlers.

For resource forms, acquisition can determine release and use computations; a bracketing model must retain that dependency. A Framework for Higher-Order Effects and Handlers, sections 4.5-4.6.

Effect4's DB-05 already makes the no-`HHandler` choice conditional on checked first-order references. `Flow.Region` currently remains empty. There is no implemented region calculus to replace. Terminating fragments may elaborate into the well-founded `Program`; arbitrary cyclic graphs still need their own relational execution account. Local evidence L2, L4.

# 6. Is a decision an operation in both systems?

**Recommendation: use one typed request-and-answer protocol while retaining who owns the choice and whether it is observable.** A request can be an operation; its replay answer is separate data.

| Kind | Treatment |
|---|---|
| Consumer or foreign input | Explicit event: call, response, settlement or abort arrival. |
| Internal permitted choice | Internal transition constrained by the current state and policy. |
| Implementation parameter | A fixed or stateful policy selected by the host profile. |
| Deterministic consequence | Compute it from prior events; do not ask an independent arbitrary choice. |

The protocol should carry a stable site/request identity, origin, answer type and a compatibility predicate against the current configuration. A tape entry supplies an answer to that request. The tape is replay evidence; it is not proof that the environment controls every internal choice.

Implementation-defined behavior may require a consistent policy over an execution. Modeling each occurrence as an unrelated fresh choice can add forbidden behaviors. Likewise, if a race winner follows from completion order, an independent winner entry must agree with that order. Promise dequeueing after enqueue order is fixed is a deterministic consequence, not another free choice.

Choice Trees distinguishes external events from internal branching and explains why exposing an internal coin flip as an ordinary visible event changes the relevant equivalence. The same distinction should survive the shared request representation and its observation masks. Choice Trees, sections 2-3 and 5.

### The finite checker and the full relation

The proposed finite checker should accept exactly the transitions admitted by the corresponding finite relation, under supplied environments and compatible answers. Prove both directions of that statement. Missing answers remain live frontiers. Malformed input, typed failure, interruption and host-profile refusal retain separate meanings.

For a deterministic fragment, prove that fixing all required compatible answers and the initial profile fixes a path. This does not prove termination, fairness, or that the tape will ever be exhausted. Keep safety over all admitted paths separate from may-complete and must-complete claims.

Effect4 already has a representative scheduler decision alphabet and finite-run relation. Its whole-machine configuration and Flow decision module remain stubs. WHATWG DB-02/DB-03 supplies design intent, not an implemented common protocol. The next work is a shared semantic contract and checked embeddings, not renaming the two existing notions and claiming agreement. Local evidence L2, L6.

## 7. What Flow-to-frame simulation is affordable?

**Recommendation: first prove a simulation for a small admitted fragment whose continuation and atom meanings are explicit.** Do not treat generator lowering as the frame transition relation by definition.

The current frame model has named continuations and thunks, first-order primitives, a continuation stack, interruption state and a bounded runner. `PrimInterp` supplies the meaning of those names. In particular, `iterNext` supplies an entire finite inline generator segment, and finalizer meaning is a supplied exit function. Async parking and scheduler execution are outside that packet. These are substantial boundaries, not a full semantics of arbitrary JavaScript generators. Local evidence L2.

Pinned rc.112 advances its iterator with `.next(value)`, folds successful exits inline, returns a yielded failure, and pushes an iterator frame when a non-exit effect is yielded. Calling this a one-shot continuation is useful operational intuition; it does not equate the runtime with a general multi-shot handler calculus. Defunctionalization explains the conversion of higher-order control into first-order tags and environments. Defunctionalization at Work. Local evidence L5.

The following candidate relation is precise enough to start a packet:

```
Related(flow block, payload, captures, region stack;
        current primitive, frame stack, mask state)

Related(c, f) and FlowStep(c, label, c')
  imply matching finite frame steps from f to f',
  equal retained observations, and Related(c', f').
```

Use separate obligations for initial-state agreement, payload/capture typing, continuation dispatch, operation answers, terminal outcomes and failure unwinding. If administrative steps may stutter, give a decreasing measure or a separate progress argument; arbitrary zero-step matching must not conceal divergence.

The lowest-risk first profile contains first-order atoms with registered semantics, sequential operations, returns and checked branches. Then add ordinary loops with an explicit relation over block re-entry. Tie `iterNext` to the generated iterator rather than leaving that assumption unexamined. Infinite inline

success loops also need treatment; a total function returning a finite segment cannot model them without a boundary or a finer step relation.

### A concrete cleanup counterexample

A fresh rc.112 probe constructed a generator with `try { yield* Effect.fail(...) } finally { ... }`. It returned failure without executing the JavaScript `finally`. An Effect scope with `acquireRelease` did execute release and passed the failure exit. The pinned iterator code explains the first result: it returns the failed yielded effect rather than resuming that generator through JavaScript exception unwinding. Local evidence L5, L7.

Consequently, a compiler must not replace Effect cleanup with JavaScript `try/finally` around yields without a separate correctness argument. Explicit scope/finalizer translation should precede any such optimization.

Leijen's selective continuation-passing compilation is a design precedent. Hillerstrom and Lindley prove agreement between a source calculus and a generalized CEK machine; that is the closer proof pattern here. OCaml's one-shot runtime demonstrates efficient continuation handling while preserving a distinct host boundary. None of these papers proves the current Flow renderer. Type Directed Compilation of Row-Typed Algebraic Effects, section 5, Liberating Effects with Rows and Handlers, Retrofitting Effect Handlers onto OCaml.

A generated dispatch loop can reduce the distance between source and target states. It still requires proofs about atom evaluation, branch dispatch, bindings, Effect execution and the observer. It does not make host execution a Lean theorem automatically.

## 8. What cleanup criterion can connect Scope and AbortSignal?

**Recommendation: model ownership, abort notification, shutdown and cleanup completion as distinct events.** Share a lifecycle vocabulary while retaining each API's obligations.

DOM abort changes a signal's state, establishes a reason, runs its registered abort algorithms and fires an event. Repeated abort leaves the first state in place. Web platform APIs using promises to represent abortable operations must reject with the abort reason and reject immediately when the supplied signal is already aborted. This does not establish that all work has stopped or all resources are released. DOM, aborting ongoing activities.

Effect's scope model closes its state before running registered finalizers and orders them last-in-first-out. Its abstract finalizer interpreter is total, so actual asynchronous finalizer progress needs additional execution semantics. Fresh host probes observed release after a body state change and failure, and observed two finalizers run in reverse order with no second execution on repeated close. Local evidence L2, L7.

| Phase | Proposed obligation |
|---|---|
| Acquisition has not succeeded | No release for an unowned resource. |
| Acquisition succeeds | Establish ownership and registration without an interruptible gap. |
| Body runs or fails | Preserve the state and exit required by release. |
| Abort is observed | Record reason/notification; apply the adapter's interruption policy. |
| Scope begins closing | Prevent duplicate close traversal and define re-entrant registration. |

| Phase | Proposed obligation |
|---|---|
| Release runs | Use its registration identity, captured environment and prescribed exit. |
| Release completes | Combine failures and restore masking according to the target contract. |

"Exactly once" needs two separate statements. **Safety:** each eligible registration identity is invoked at most once over the scope's lifetime, including repeated, concurrent and re-entrant close calls. **Progress:** closing reaches and completes the entry, assuming preceding finalizers and the entry itself terminate and the relevant scheduler/foreign operations make progress. Completion may be failure. Re-registration, replacement by key and registration after close each need explicit rules.

For the bridge, retain at least `abort observed`, `closing`, `release invoked`, `release completed` and terminal outcome as internal events. The chosen public mask may hide some, but the resource theorem must still inspect them. A single promise rejection cannot prove cleanup completion.

Structured-concurrency practice supports the need to protect asynchronous cleanup from outside cancellation. Trio, for example, allows a shielded cleanup scope while still allowing its own timeout. This is a design comparison, not a claim that Trio's cancellation semantics equal Effect's. Trio cancellation and shielding.

The next resource battery should cover failed acquisition, interruption between acquisition and registration, body failure after mutation, repeated close, re-entrant add, release failure, a nonterminating finalizer, and abort racing with completion. Include loser cleanup before declaring a race adapter complete. Preserve the pinned target's failure precedence instead of imposing one universal bracket equation across APIs.

# 9. What do the real programs use?

**The fresh scan supports generators, layers and error handling as early priorities, with ordinary loops and resource boundaries close behind.** Its counts describe a selected corpus, not the prevalence of Effect features in all applications or in rc.112 alone.

The pin list contains 32 rows and 31 unique names: one duplicate and one absent repository. All 30 present selected repositories were clean and matched their recorded commits before and after scanning. Four additional on-disk repositories were excluded from that selection. There were 25,645 tracked TypeScript-family files; removing declaration files and a vendored Effect subtree left 24,401 analysis files. TypeScript 5.9.2 parsed every selected file; 35 completion-fixture files with syntax diagnostics were excluded from AST counts. Local evidence L10.

The scanner resolves lexical import bindings, aliases, simple constant aliases/destructuring, literal computed properties and shadowing. It counts both explicit calls and references, because a bare combinator passed into `pipe` is still a use.

| Family | References | Calls | Repositories |
|---|---|---|---|
| Layer members | 9,716 | 9,574 | 26 |
| Scope members | 345 | 276 | 13 |
| Effect resource operations | 1,415 | 1,109 | 22 |
| Effect fork variants | 573 | 195 | 18 |

| Family | References | Calls | Repositories |
|---|---|---|---|
| Effect race variants | 31 | 31 | 8 |
| Effect catch variants | 7,741 | 7,740 | 26 |
| Effect.gen | 29,367 | 29,367 | 27 |

Resource operations here mean the explicit set `scoped`, `acquireRelease`, `acquireUseRelease`, `acquireReleaseInterruptible`, `addFinalizer`, `ensuring`, `onExit` and `onInterrupt`. Counts are source sites, not execution frequencies. The parent independently recomputed all seven aggregate families from the stored rows.

The leading Layer references were `effect` 3,908; `succeed` 2,063; `mergeAll` 1,348; `provide` 1,048; and `provideMerge` 514. `catchTag` supplied 6,165 catch references. Forks were dominated by `forkChild` 254 and `forkScoped` 245. Counting calls alone would lose many uses: `scoped` had 455 references but 149 explicit calls. Local evidence L10.

## Generators and control flow

Of 29,367 generator calls, 29,365 had an inline generator function. Among those bodies, 1,016 contained direct loops and 4,456 contained a nested `Effect.gen`. Direct loops totaled 1,422: 1,141 `for...of`, 171 ordinary `for`, 87 `while` and 23 `do...while`. There were 707 bodies yielding inside a direct loop. Syntactic generator nesting reached depth four.

Counting loops inside nested callbacks raises the body count to 1,420, but that is lexical containment rather than a loop in the enclosing control flow. One real body had two labelled `continue` statements. There were 82 bodies with a direct `finally`; their presence is a reason to inspect semantics, not proof that they implement Effect cleanup. Local evidence L10.

## What the denominator changes

Paths marked as tests, fixtures, examples or benchmarks accounted for 21,311 generator calls. The remaining path stratum had 8,056 calls, including 558 inline bodies with direct loops. Alchemy alone contributed 15,834 generator calls and 6,612 catch references. Excluding it leaves 13,533 generator calls and 1,129 catch references. Neither path filtering nor removing one repository produces a representative production sample.

The corpus mixes rc.112, nearby release candidates, earlier v4 betas, v3, v2 and older packages. Dependency declarations are not uniformly resolved per-file runtime versions. Unsupported old imports, cross-file wrappers/re-exports, dynamic imports, mutable aliases and `Effect.fn` generators are outside the recognizer. Zero detected modern calls therefore does not mean zero Effect use. Duplicate and generated code were not deduplicated. Local evidence L10.

**No reducibility result follows.** This scan constructs no control-flow graph, dominance relation, exceptional/resumption edges or strongly connected components. It supports starting with structured loops; it cannot prove every admissible graph is reducible. Keep a general dispatch route available in the design until a separate graph analysis justifies a restriction.

For the next iteration, stratify by admitted runtime version and repository role, recognize `Effect.fn`, then build graphs only for candidate lowering inputs. Measure accepted/rejected shapes and reasons before treating raw frequency as a release criterion.

# 10. What do Streams' masks constrain?

**Recommendation: prove output agreement and resource discipline separately.** M1 can retain chunk order and outcome while hiding excessive prefetching or queue growth. M2 exposes more ordering, but a backpressure theorem still needs the relevant calls and state.

The pinned `ReadableStreamPipeTo` requirements prohibit initiating a read when the destination writer's desired size is nonpositive or null, and prohibit new reads after shutdown starts. Already-issued reads can still complete. The standard also recommends avoiding unnecessary serialization behind each completed write. Thus backpressure is neither unrestricted prefetching nor a universal one-chunk pipeline. Streams, pinned piping requirements.

Start with the full-state invariant:

```
Pipe initiates read
  implies current writer desiredSize is positive
  and shutdown has not begun.
```

Then prove an M1 chunk/outcome theorem as another projection of the same runs. The local DB-05 phrase about satisfying piping under M1 needs this additional invariant: output observations alone cannot express every piping requirement. Local evidence L6.

A host probe set high-water mark to 1 and enqueued three unit-sized chunks. It observed desired size -2. The standard admits overfull queues and an infinite high-water mark. Tee can also keep pulling for one branch while another accumulates data. These defeat an unconditional memory-cap claim. Streams, controller desired size, Streams, high-water-mark validation. Local evidence L7.

### A candidate bounded law

For a count/size queue, assume initial occupancy `q0 <= H`, finite nonnegative threshold `H` and chunk bound `C`, a positive bound `k` on all outstanding admitted additions including batches, admission only while below threshold, additive nonnegative accounting, and no other enqueue path. Then propose proving `q <= H + k*C` at every reachable state.

This is a proposed invariant with explicit premises. It bounds accounted queue size, not total memory: zero-sized entries, hidden buffers and retained objects can defeat a memory interpretation. Generalizing it to the standard's binary64 size arithmetic requires separate numerical obligations.

Do not optimize away jobs merely because they look administrative. The pinned tee algorithm deliberately delays work through a microtask to order successful reads against asynchronous errors. A fusion theorem needs the selected mask, callback effects, error precedence, locking, shutdown and queue discipline. Streams, pinned tee algorithm.

Yang and Wu supply a way to derive sufficient conditions for effect equations to survive handler composition. Their fusion technique is useful for stating the clauses to prove; it does not establish arbitrary Stream map fusion, `pipeThrough` associativity or tee laws. Reasoning about Effect Interaction by Fusion.

# Recommended work order

The original priority of questions 1, 3 and 6 remains justified, with a narrower deliverable: freeze the vocabulary of jobs, completion and decisions before building a universal scheduler. The following packets are proposed research outcomes, not implementation dispatches.

| Order | Concrete result and stopping condition |
|---|---|
| 1. Shared boundary | Define a single-agent host profile, job/promise operations, typed choice protocol and exact M1/M2 event projections. Stop when competing interpretations and minimal counterexamples have been resolved. |
| 2. Error and membership | Freeze completion propagation, catch delimitation, per-method target errors, row normal form and duplicate identity. Close the independent batteries and the outstanding Effects assurance work. |
| 3. First lowering proof | Admit one operation family, pure registered atoms, sequential execution and branches. Prove the source/frame relation and run matched target observations. |
| 4. Loops and resources | Add ordinary loops, labelled exits, captured environments and explicit release. Require failure/cancellation/cleanup counterexamples and divergence boundaries. |
| 5. Streams bridge | Realize piping requirements over full runs, prove M1 output agreement plus backpressure, and then prove the chosen M2 obligations under a specific host profile. |

Before expanding a packet, require a concrete missing behavior or a corpus-derived admitted use case. Stop a search when primary evidence resolves the factual question and the remaining issue is a formal contract choice. Do not substitute more literature for choosing and proving that contract.

The existing Lean performance study gives one useful implementation warning: repeatedly retaining a fresh alias while mutating a byte array can dominate execution cost. Its Windows measurements were 0.29 ms for 100,000 unique-array writes and 257.01 ms when a fresh live alias was retained per iteration. These are prior recorded measurements, not rerun results or scheduler costs. Benchmark actual queue replay and trace retention on the intended host. The study's earlier axiom-policy restrictions also predate WHATWG's current R-11 ruling. Local evidence L11.

# Local evidence and verification

The evidence bundle is retained in the Effect4 checkout under `docs/research/2026-09-02-shared-semantics/`. It contains source identities, the corpus inventory and compressed site records, scanner and fixture, row experiments, exploratory Lean witnesses, host probes, and command receipts. The report does not replace the owning design documents.

**L1 - Current package state.** Root `/Users/pooks/Dev/lean4-effect4`: `lakefile.toml`, `.lake/package-overrides.json`, `lake-manifest.json`, `PLAN.md` current-phase section, `PORT-MANIFEST.md` authority pins, `docs/ARCHITECTURE.md`, and `docs/research/2026-09-02-ecosystem-audit.md` sections 10-11. Effects root `/Users/pooks/Dev/lean4-effects`: `docs/CLAIM-BOUNDARY.md`, current v0.2.0 boundary. Existing unrelated untracked files were retained.

**L2 - Runtime and semantics.** Effect4: `Effect4/Runtime/Runtime.lean` (`Prim`, `PrimInterp`, `FrameFiber`), `Effect4/Runtime/Scope.lean`, `Effect4/Semantics/Cause.lean`, `Effect4/Concurrency/Scheduler.lean`, `docs/FRAMES-DAG.md`, `docs/SCOPE-DAG.md`, and the configuration/step/decision/region stubs. Source declarations take precedence over old RED/stub status prose when reporting what exists; graph closure is a separate claim.

**L3 - Lowering.** Effect4: `Effect4/Target/TypeScript/EffectV4.lean` (`OpRow`, `methodType`, `Script`, `lower`), `Effect4/Target/TypeScript/Simulation.lean`, and `Effect4/Meta/Derive.lean`. No source-to-host simulation theorem was found in the empty simulation module.

**L4 - Prior rulings.** Effect4 `docs/DESIGN-BASIS.md`, DB-02 through DB-07. Foldlab root `/Users/pooks/Dev/foldlab`: `.staging/algebraic-review/prog-carrier.md`, `handlers-semantics.md`, `THE-ALGEBRA.md`; `docs/SPECS.md` decision 9; `.staging/operational-structure/EFFECT-AST-PLACEMENT.md`. The older placement study used a different sample and method.

**L5 - Pinned host code.** Effect4 `vendor/effect-4.0.0-rc.112/src/internal/effect.ts`, iterator and scope-close definitions; `src/internal/core.ts`, failure propagation; `src/Scheduler.ts`, priority buckets and mixed dispatch. `internal/effect.ts` and `Scheduler.ts` were hash-equal to the installed package sources used by the host probes. Source files in this vendor subset are not a complete runnable package.

**L6 - WHATWG local contract.** Root `/Users/pooks/Dev/lean4-whatwg`: `docs/DESIGN-BASIS.md` DB-02 through DB-05; `docs/REIFICATION-STRATEGY.md` strata and RS-Q1 through RS-Q5; `Whatwg/Streams/Semantics/` stubs; pinned `vendor/whatwg-streams-b9ba9f49/index.bs`. The specification file SHA-256 is `24360b4f8446e6c80e185c5021fcca9b67a7e0bb62490a00109080ebc04c6440`.

**L7 - Fresh finite host evidence.** Node v22.23.2, installed `effect@4.0.0-rc.112`. Five assertions checked generator failure versus JavaScript finally, exit-aware release after state change, LIFO/idempotent scope close, repeated abort with a stable first reason, and overfull stream desired size. A sixth probe compared direct and extra-reaction promise ordering. These are finite Node observations, not browser conformance or universal proofs.

**L8 - Fresh exploratory Lean witnesses.** Nine statements checked returned-error recovery, ambient failure, throw/bind, catch delimitation, duplicate service identity, codiagonal collapse, cause order and squash information loss. Eight axiom receipts were empty; `throw_absorbs` used `Quot.sound`. These research witnesses are not a newly frozen library contract or cutover acceptance battery.

**L9 - Row measurements.** `rows/measurements.json`, `extended_measurements.json`, `raised_measurements.json`, generating scripts and exact Lean inputs retain all trials, failures and settings. Lean 4.33.1, Effects `fa3cdc96`, macOS arm64. Three processes per successful measured variant; startup/import time included. No production workload or memory benchmark was run.

**L10 - Corpus.** `corpus/selected-pins.tsv` has SHA-256 `99ae9a9ee91dea505d36114d091c4c3f55d5be81cebc2930afe01c32db34932a`. The ordered path/content digest is `41ceead9eccc4fac95cdd626a6fe126e92c7fe0df1273dbc4bc999126b3416ed`. `scan.cjs` SHA-256 is `07f82ea189b78ca8713cfb3e39b76ec4f45f78a9b58223a039889606bc7793bb`. Two full scans agreed on source digest; the parent independently checked all group aggregates. Synthetic alias/shadowing and loop fixtures passed. No CFG analysis or cross-project typecheck was performed.

**L11 - Prior performance measurements.** WHATWG `docs/research/2026-09-01-lean-stdlib-strategy-and-performance.md`, section 2.2. Windows 11, Ryzen 7 8700F, Lean 4.33.1, median of three, as recorded in that report. Current axiom policy is in WHATWG `AGENTS.md`, Gates, R-11.

## Verification performed for this report

`lake build Effect4.Runtime.Runtime Effect4.Runtime.Scope` exited zero. Direct Lean runs of `FramesContract.lean`, `FramesAxiomReport.lean` and `ScopeContract.lean` exited zero. All 149 printed frame theorem receipts stayed within `propext` and `Quot.sound`. The nine exploratory witnesses and all finite host assertions passed. These commands reported the existing stale dependency-manifest warning. No full repository build, trust gate, runtime-coverage gate or browser suite is claimed.

Primary sources and the highest-impact claims were checked during research. Failed retrievals were bounded: an arXiv HTML rendering and an institutional PDF link failed; available primary paper copies or the institution's publication record supplied the narrower claims used. No paper implementation was rebuilt. Standards other than the pinned Streams source were consulted as living text on 2 September 2026; publication dates below identify the research papers, not the access date.

Document checks covered text extraction, embedded citation links and page margins throughout the PDF. Visual inspection sampled seven pages, including the opening page, tables, proof sketch, Streams analysis and source record.

# Source reading map

The following reading order groups the papers by the decision they inform. Links near the analysis support the corresponding claims; these publication details provide provenance.

- **Scheduling:** Matthew C. Loring, Mark Marron and Daan Leijen, *Semantics of Asynchronous JavaScript*, Microsoft Research technical report, 26 July 2017 / DLS 2017. Use the parameterized scheduler and stated abstraction limits.
- **Equivalence:** Li-yao Xia and coauthors, *Interaction Trees*, POPL 2020; Nicolas Chappe, Paul He, Ludovic Henrio, Yannick Zakowski and Steve Zdancewic, *Choice Trees*, arXiv version 1, 2022; Irene Yoon, Yannick Zakowski and Steve Zdancewic, *Formal Reasoning about Layered Monadic Interpreters*, ICFP 2022. Read weak equivalence, internal branching and layer-specific laws together.
- **Errors:** Gordon Plotkin and Matija Pretnar, *Handling Algebraic Effects*, LMCS 2013. Pair it with Web IDL's distinction between exception objects, throwing and promise rejection.
- **Rows:** Wouter Swierstra, *Data Types a la Carte*, JFP 2008; Oleg Kiselyov and Hiromi Ishii, *Freer Monads, More Extensible Effects*, Haskell 2015; Daan Leijen, *Koka: Programming with Row-Polymorphic Effect Types*, 2014; Ningning Xie and Daan Leijen, *Generalized Evidence Passing for Effect Handlers*, ICFP 2021. Separate inclusion, removal, duplicate labels and runtime evidence.
- **Scoped syntax:** Nicolas Wu, Tom Schrijvers and Ralf Hinze, *Effect Handlers in Scope*, Haskell 2014; Maciej Pirog, Tom Schrijvers, Nicolas Wu and Mauro Jaskelioff, *Syntax and Semantics for Operations with Scopes*, LICS 2018; Roger Bosman, Birthe van den Berg, Wenhao Tang and Tom Schrijvers, *A Calculus for Scoped Effects and Handlers*, LMCS 2024.
- **Deferred and modular elaboration:** Birthe van den Berg, Tom Schrijvers, Casper Bach-Poulsen and Nicolas Wu, *Latent Effects for Reusable Language Components*, APLAS 2021; Casper Bach Poulsen and Cas van der Rest, *Hefty Algebras*, POPL 2023; Birthe van den Berg and Tom Schrijvers, *A Framework for Higher-Order Effects and Handlers*, 2023 preprint.
- **Machines and compilation:** Olivier Danvy and Lasse R. Nielsen, *Defunctionalization at Work*, BRICS 2001; Daan Leijen, *Type Directed Compilation of Row-Typed Algebraic Effects*, POPL 2017; Daniel Hillerstrom and Sam Lindley, *Liberating Effects with Rows and Handlers*, TyDe 2016; KC Sivaramakrishnan and coauthors, *Retrofitting Effect Handlers onto OCaml*, PLDI 2021; Ohad Kammar, Sam Lindley and Nicolas Oury, *Handlers in Action*, ICFP 2013, as a broader operational introduction.
- **Composition:** Zhixuan Yang and Nicolas Wu, *Reasoning about Effect Interaction by Fusion*, ICFP 2021. Apply its sufficient conditions to chosen handler clauses rather than assuming a general stream fusion law.

The remaining uncertainty is concentrated in formalization and admission: exact M2 events, cause projection, instance identity, callback semantics, fairness premises and graph reducibility. The research supplies evidence and falsifiers for deciding those points; it does not turn the proposed decisions into established theorems.